

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ЛАБОРАТОРНАЯ РАБОТА

по теме

Объектно-ориентированное проектирование системы. Язык UML

Выполнил
студент гр. 250541

Бойцов А.В.

Проверил
ассистент каф. ЭВМ

Богдан Е.В.

Минск 2026

СОДЕРЖАНИЕ

1 ВИДЕНИЕ И ГРАНИЦЫ ПРОЕКТА.....	3
2 ДИАГРАММА ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ.....	4
3 ДИАГРАММА АКТИВНОСТИ.....	6
4 ДИАГРАММА ПОСЛЕДОВАТЕЛЬНОСТИ.....	11
5 ДИАГРАММА РАЗВЕРТЫВАНИЯ.....	12
6 ДИАГРАММА КЛАССОВ.....	14
7 ДИАГРАММА СОСТОЯНИЙ.....	17

1 ВИДЕНИЕ И ГРАНИЦЫ ПРОЕКТА

Проект «Монополия» представляет собой цифровую реализацию классической экономической настольной игры, ориентированную на обучение стратегическому мышлению, управлению ресурсами и принятию решений в условиях ограничений. Система должна обеспечить интерактивный игровой процесс, в котором игроки перемещаются по игровому полю, совершают сделки, покупают и улучшают собственность, платят ренту и взаимодействуют с особыми клетками.

Цель проекта — создать структурированную, расширяемую и архитектурно чистую платформу, построенную на строгой UML-модели и ER-структуре, обеспечивающую:

- корректную игровую механику,
- предсказуемое поведение объектов,
- возможность дальнейшего расширения (новые типы клеток, режимы игры, ИИ),
- удобство сопровождения и модификации кода,
- прозрачную архитектуру для анализа и обучения.

Система должна быть достаточно простой для освоения новичками и достаточно гибкой для использования в учебных целях (ООП, UML, ER-моделирование, архитектура ПО).

В проект первой версии входит:

- **Игровой цикл:** бросок кубиков, перемещение, выполнение действий клетки.
- **Управление игроками:** создание, хранение состояния, смена хода.
- **Покупка собственности:** PropertyCell, RailRoadCell, UtilityCell.
- **Рента:** начисление, списание, передача между игроками.
- **Улучшения:** покупка домов (numHouses).
- **Специальные клетки:** GoToPrisonCell.
- **ER-модель и UML-архитектура:** строгая структура данных.
- **Базовый UI:** кнопки действий, отображение состояния.
- **Карточки «Шанс» и «Общественная казна».**

В проект не входит в проект:

- Онлайн-мультиплеер.
- Сохранение/загрузка партий.
- Аукционы.
- ИИ-игроки.
- Несколько игровых полей.
- Кастомизация правил.
- Экономические модификаторы (налоги, инфляция).
- Графическая анимация.
- Сетевая синхронизация.
- Продажа собственности: PropertyCell, RailRoadCell, UtilityCell.

2 ДИАГРАММА ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ

Диаграмма вариантов использования описывает функциональность системы с точки зрения пользователей (акторов) и их взаимодействия с системой через варианты использования (use cases). Для игры Монополия определены два основных актора: Игрок и Банк(Система). \

1. Игрок (Player)

Это внешний пользователь системы. Он инициирует все игровые действия, связанные с процессом партии.

Связан с вариантами использования:

- Начать игру
- Бросить кубики
- Передвинуть фишку
- Действие на клетке
- Покупка собственности
- Оплата ренты
- Торговать
- Завершить игру

2. Банк / Система (Bank / System)

Это внутренний актер, представляющий автоматическую игровую логику:

- начисляет ренту,
- проверяет возможность покупки,
- выполняет действия клетки,
- контролирует правила.
- Связан с вариантами использования:
- Действие на клетке
- Оплата ренты
- Покупка собственности

Ниже — описание каждой активности на диаграмме.

1. Начать игру

Игрок запускает новую партию. Система подготавливает поле, игроков, баланс, позиции.

2. Бросить кубики

Игрок инициирует бросок. Система генерирует случайное число (1–12).

3. Передвинуть фишку

Игрок перемещает свою фишку на количество шагов, выпавших на кубиках. Система фиксирует новую позицию.

4. Действие на клетке

Система определяет тип клетки:

- Собственность;
- Налог;
- Тюрьма;

- Шанс;
- Общественная казна;
- Железная дорога;

И выполняет соответствующее действия:

5. Покупка собственности

Если игрок попал на свободную клетку собственности, он может её купить. Система списывает деньги и назначает владельца.

6. Оплата ренты

Если клетка принадлежит другому игроку — система начисляет ренту. Игрок платит владельцу.

7. Торговать

Игрок может обмениваться собственностью с другими игроками. Система проверяет корректность сделки.

8. Завершить игру

Игрок завершает партию. Система подводит итог: победитель, активы, статистика.

Указанные варианты использования сведены в диаграмму вариантов использования, показанную на рисунке 1.

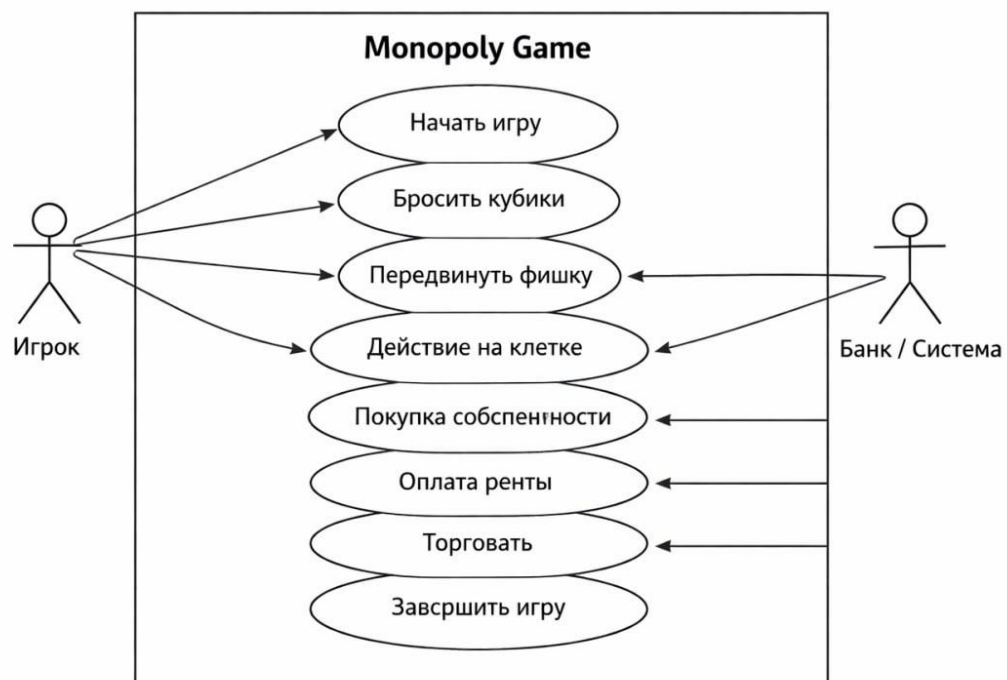


Рисунок 1 – Диаграмма вариантов использования

3 ДИАГРАММА АКТИВНОСТИ

Диаграмма активности описывает последовательность действий и процессов в системе. Для игры Монополия выберем три ключевых сценария, отображающих основной функционал системы. На диаграмме показаны начальные и конечные точки, действия, условия ветвления и параллельные процессы.

Сценарий 1: процесс проверки, способен ли игрок оплатить ренту при попадании на чужую собственность.

Система выполняет:

- вычисление суммы ренты;
- проверку текущего баланса игрока;
- сравнение баланса и требуемой суммы.

Ветка “Choose == 1” → Действие: “Списываем деньги”

Если у игрока достаточно средств:

- система уменьшает баланс игрока на сумму ренты;
- увеличивает баланс владельца клетки;
- фиксирует транзакцию.

Это action node, завершающий успешный сценарий.

Ветка “Choose == 0” → Действие: “Игрок банкрот”

Если средств недостаточно:

- игрок объявляется банкротом;
- игрок исключается из игры.

Это альтернативный сценарий.

Сценарий 1 отражен на рисунке 2.

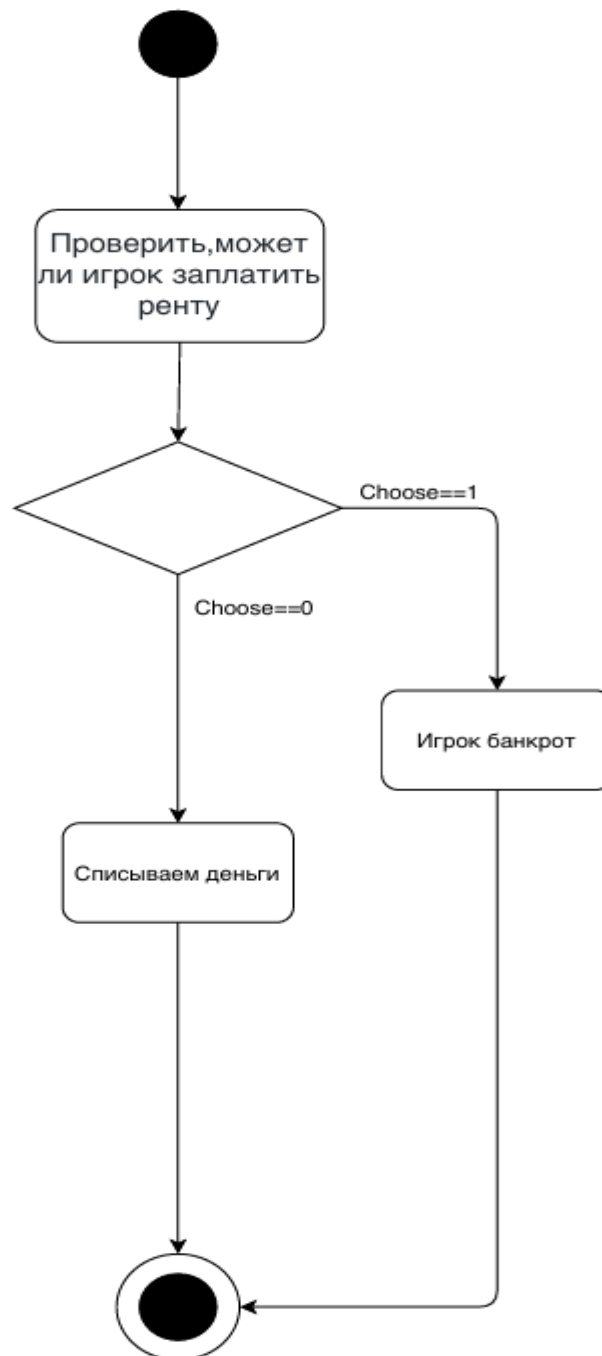


Рисунок 2 – Сценарий процесса проверки, способен ли игрок оплатить ренту при попадании на чужую собственность

Сценарий 2: попадание игрока в зону “Тюрьма”

Диаграмма описывает логику хода игрока, когда он:

– делает выбор (например, заплатить штраф или пропустить ход),

– в зависимости от выбора либо платит штраф, либо бросает кубики,
– в зависимости от результата броска либо пропускает ход, либо делает
ход.

Сценарий 2 отражен на рисунке 3.

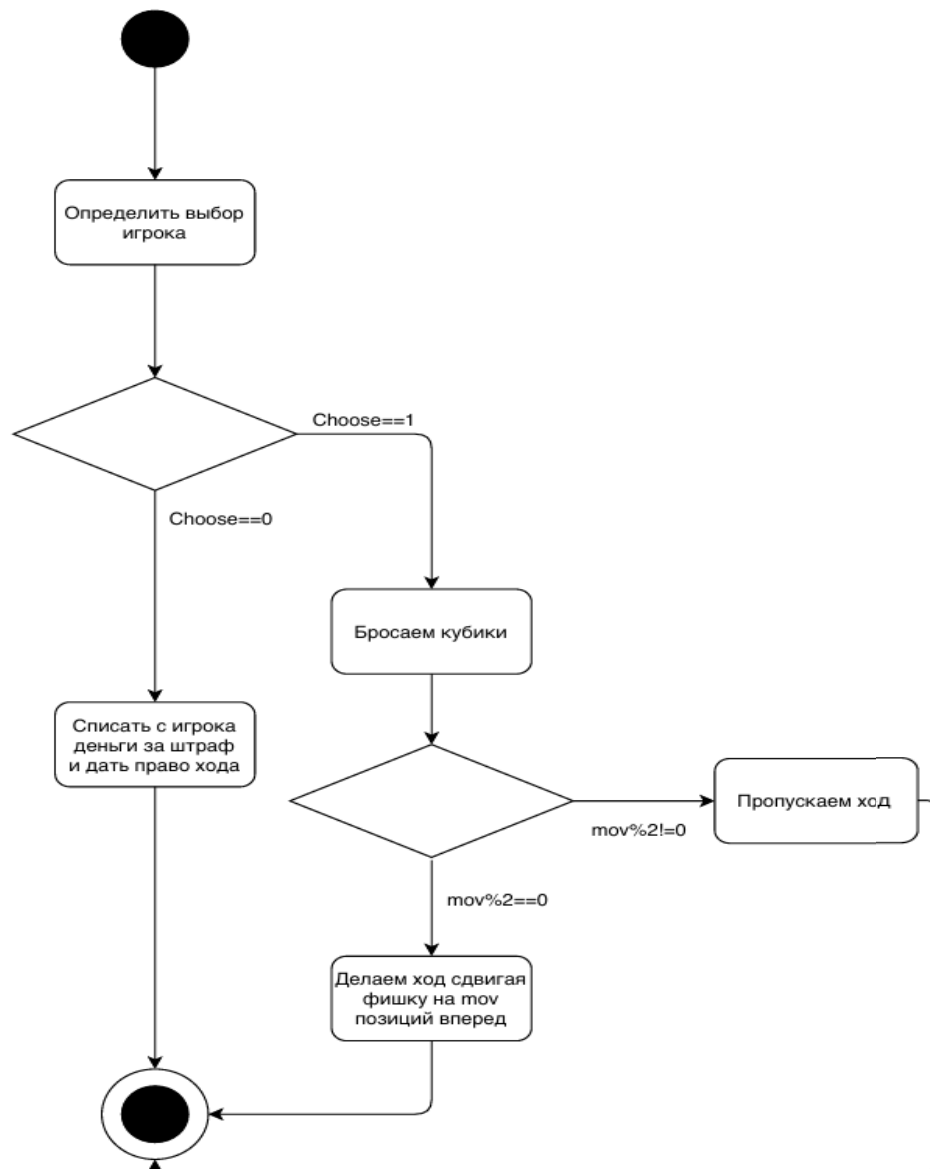


Рисунок 3 – Сценарий попадания игрока в зону “Тюрьма”

Сценарий 3: проверка баланса игрока.

Диаграмма моделирует финансовую операцию игрока, включающую два сценария:

- Игрок получает деньги.
- Игрок должен заплатить деньги, и система проверяет, может ли он это сделать.

Это локальная часть игровой логики, связанная с балансом игрока и возможным банкротством.

Если игрок должен заплатить:

- система сравнивает баланс игрока с требуемой суммой;
- результат проверки передаётся в следующий узел.

Это аналогично проверке ренты, но более универсально.

Если средств недостаточно:

- игрок объявляется банкротом;
- он исключается из игры.

Это финальный негативный сценарий.

Сценарий 3 отражен на рисунке 4.

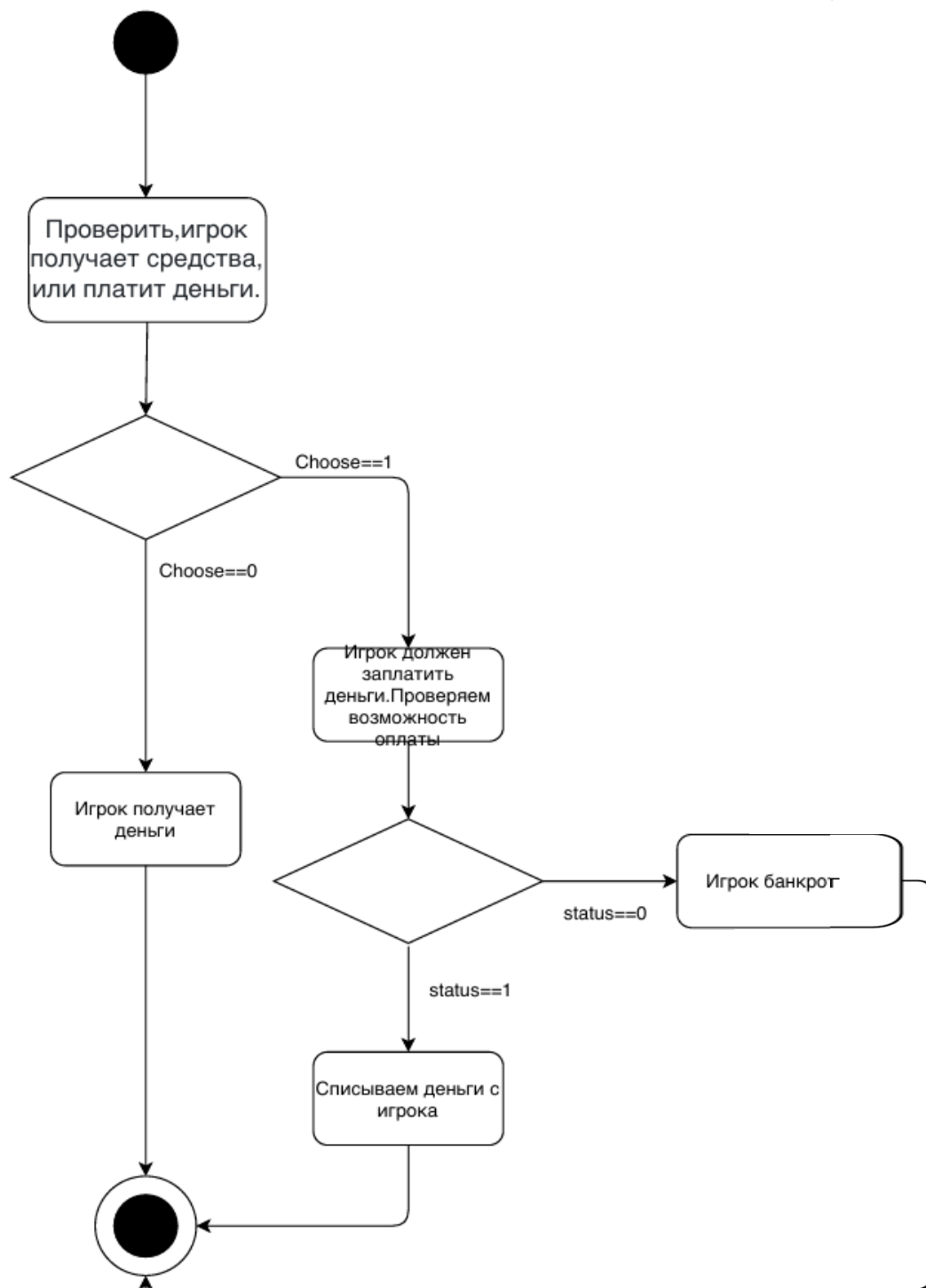


Рисунок 4 – Сценарий проверки баланса игрока

4 ДИАГРАММА ПОСЛЕДОВАТЕЛЬНОСТИ

Диаграмма последовательности для сценария авторизации отображает развития сценария механик игры в приложении

На рисунке 5 представлена диаграмма последовательности поведения игрока при взаимодействии с разными типами клеток в игре.

Диаграмма описывает **все основные игровые сценарии**, которые происходят после того, как игрок попадает на клетку:

- покупка поля
- улучшение поля
- попадание на поле противника
- попадание в тюрьму
- попадание на клетку «вопрос»

Каждый сценарий включает:

- запрос от игрока
- проверку условий системой
- выполнение действия
- возврат результата игроку

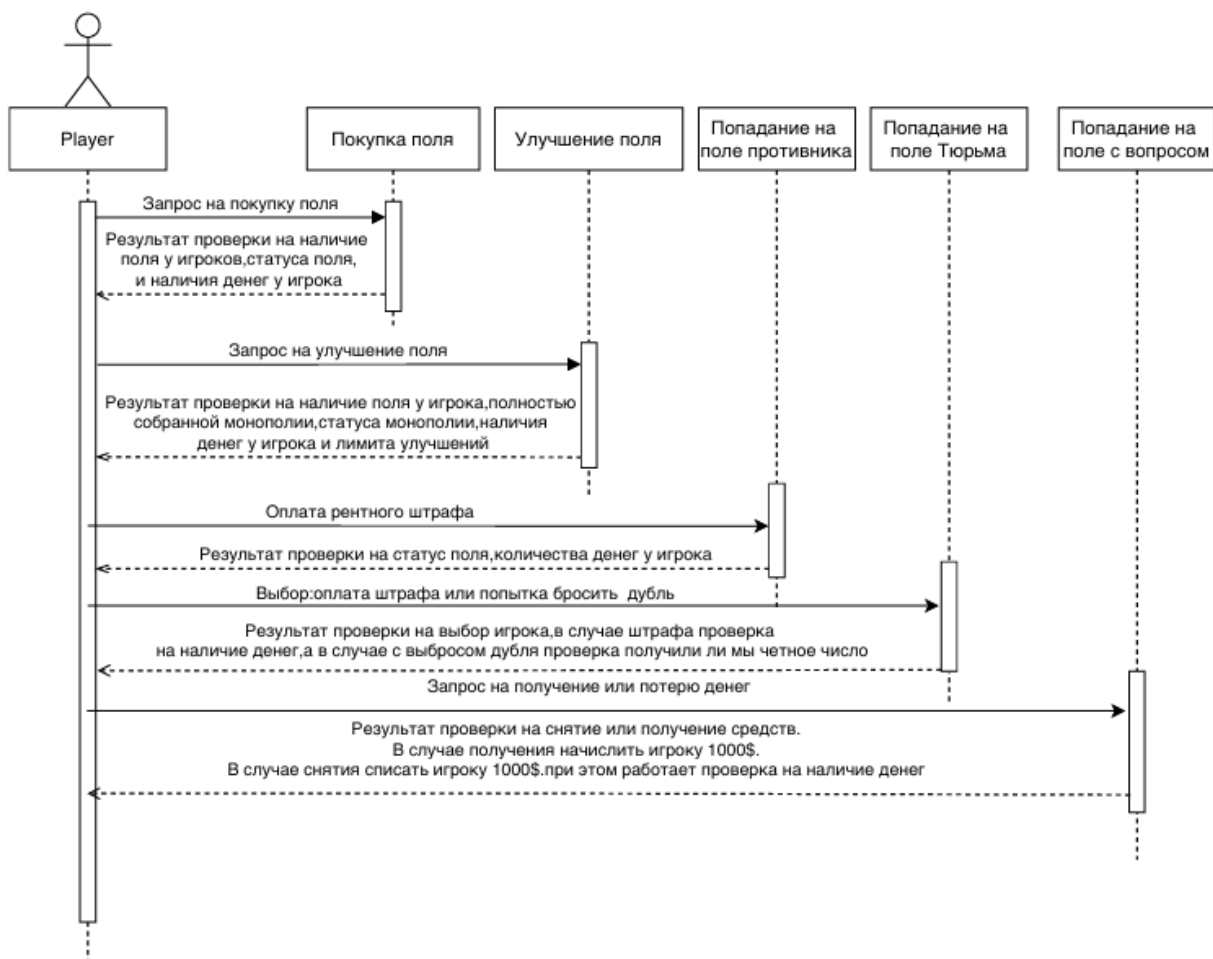


Рисунок 5 – Диаграмма последовательности действий игрока

5 ДИАГРАММА РАЗВЕРТЫВАНИЯ

Диаграмма компонентов отражает архитектурную структуру приложения «Монополия» на уровне модулей и исполняемых единиц. Она показывает, какие компоненты входят в систему, как они размещены на устройстве пользователя и каким образом взаимодействуют между собой.

1. Узел «РС» (устройство пользователя)

На диаграмме представлен узел:

<<device>> РС — персональный компьютер пользователя.

Этот узел является физической платформой, на которой запускается приложение.

Он содержит исполняемый файл:

<<executable>> Monopoly.jar

Файл Monopoly.jar — это упакованное Java-приложение, включающее все необходимые классы, ресурсы и логику игры.

2. Компонент «Player»

Обозначен как <<Component>> **Player**

Этот компонент отвечает за:

- хранение данных игрока (имя, баланс, позиция);
- выполнение действий игрока;
- взаимодействие с игровыми объектами;
- передачу данных в игровой модуль.

Компонент Player является логическим модулем, который инкапсулирует поведение и состояние каждого участника игры.

3. Компонент «Game» Обозначен как **Game**

Этот компонент представляет собой **центральный модуль игрового процесса**, включающий:

- игровое поле;
- логику ходов;
- обработку событий;
- управление состоянием игры;
- взаимодействие с интерфейсом.

Над компонентом указано «**Отображение приложения**»

Это означает, что компонент Game отвечает за визуализацию состояния игры в пользовательском интерфейсе.

Данная архитектура показана на диаграмме развертывания на рисунке 6.

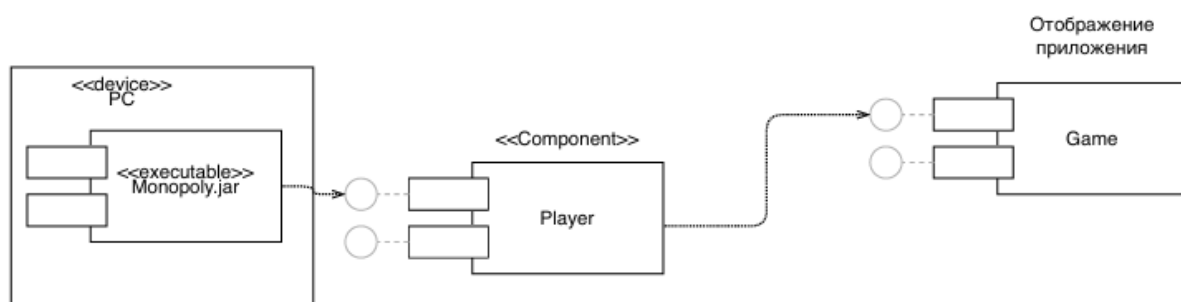


Рисунок 6 – Диаграмма развертывания игры

6 ДИАГРАММА КЛАССОВ

Диаграмма классов описывает архитектуру объектной модели игры “Монополия”.

Она показывает:

- основные сущности игры,
- их свойства,
- методы,
- наследование,
- связи между объектами.

Это — центральная UML-диаграмма, определяющая всю игровую логику.

1. Класс GameMaster

Назначение: центральный управляющий компонент, координирующий игровой процесс.

Атрибуты:

- maxPlayers : int — максимальное количество игроков.
- gameBoard : List<Cell> — коллекция клеток игрового поля.

Методы:

– playerMoved(player : Player) — обработка события перемещения игрока.

– movePlayer(player : Player, steps : int) — перемещение игрока на указанное число клеток.

– rollDice() : int — генерация результата броска кубиков.

– sendToPrison(player : Player) — отправка игрока в тюрьму.

– startGame() — инициализация игровой партии.

– reset() — сброс состояния игры.

– Методы обработки пользовательских действий:

- btnDrawCardClicked()
- btnEndTurnClicked()
- btnGetOutOfPrisonClicked()
- btnPurchasePropertyClicked()
- btnRollDiceClicked()

Роль в системе: GameMaster является координатором всех игровых событий и взаимодействует с игроками и клетками поля.

– Агрегация: GameMaster содержит коллекцию List<Cell>, представляющую игровое поле.

– Ассоциации: GameMaster взаимодействует с Player и Dice. Player взаимодействует с клетками через методы playAction().

2. Класс Player

Назначение: представление игрока, его состояния и действий.

Атрибуты:

- name:String — имя игрока.
- money:int — текущий баланс.
- position:int — позиция на игровом поле.

Методы:

- canBuyHouse() : boolean — проверка возможности покупки дома.
- exchangeProperty() — обмен собственности.
- getMonopolies() : List<PropertyCell> — получение списка монополий

игрока.

- getOutOfPrison() — выход из тюрьмы.
- purchaseProperty() — покупка собственности.
- purchaseRailRoad() — покупка железной дороги.
- purchaseHouse() — покупка дома.
- payRentTo(player : Player) — оплата ренты.
- trade(player : Player) — торговля.
- buyProperty() — универсальный метод покупки.

Роль в системе: Player инкапсулирует состояние игрока и предоставляет интерфейс для выполнения игровых действий.

3. Класс Dice

Назначение: генерация случайных значений для моделирования броска кубиков.

Методы: getRoll() : int — возвращает результат броска.

4. Абстрактный класс Cell

Назначение: базовый класс для всех типов клеток игрового поля.

Особенности:

- определяет общий интерфейс для всех клеток;
- используется в коллекции gameBoard класса GameMaster.

Наследование: Все специализированные клетки наследуются от абстрактного класса Cell. Это обеспечивает полиморфизм при вызове метода playAction().

5. Класс PropertyCell

Назначение: клетка собственности (улица).

Методы:

- playAction(player : Player) — обработка попадания игрока.
- getRent() / setRent() — работа с рентой.
- getPrice() / setPrice() — работа с ценой.
- getNumHouses() / setNumHouses() — количество домов.
- setHousePrice() — стоимость улучшения.

6. Класс RailRoadCell

Назначение: клетка железной дороги.

Методы:

- playAction(player)
- getRent()
- setPrice()
- setBaseRent()

7. Класс UtilityCell

Назначение: клетка коммунальной службы.

Методы:

- playAction(player)
- setPrice()
- getRent()
- getPrice()

8. Класс GoToPrisonCell

Назначение: клетка «Отправляйтесь в тюрьму».

Методы: playAction(player) — отправляет игрока в тюрьму.

Диаграмма классов приведена на рисунке 7.

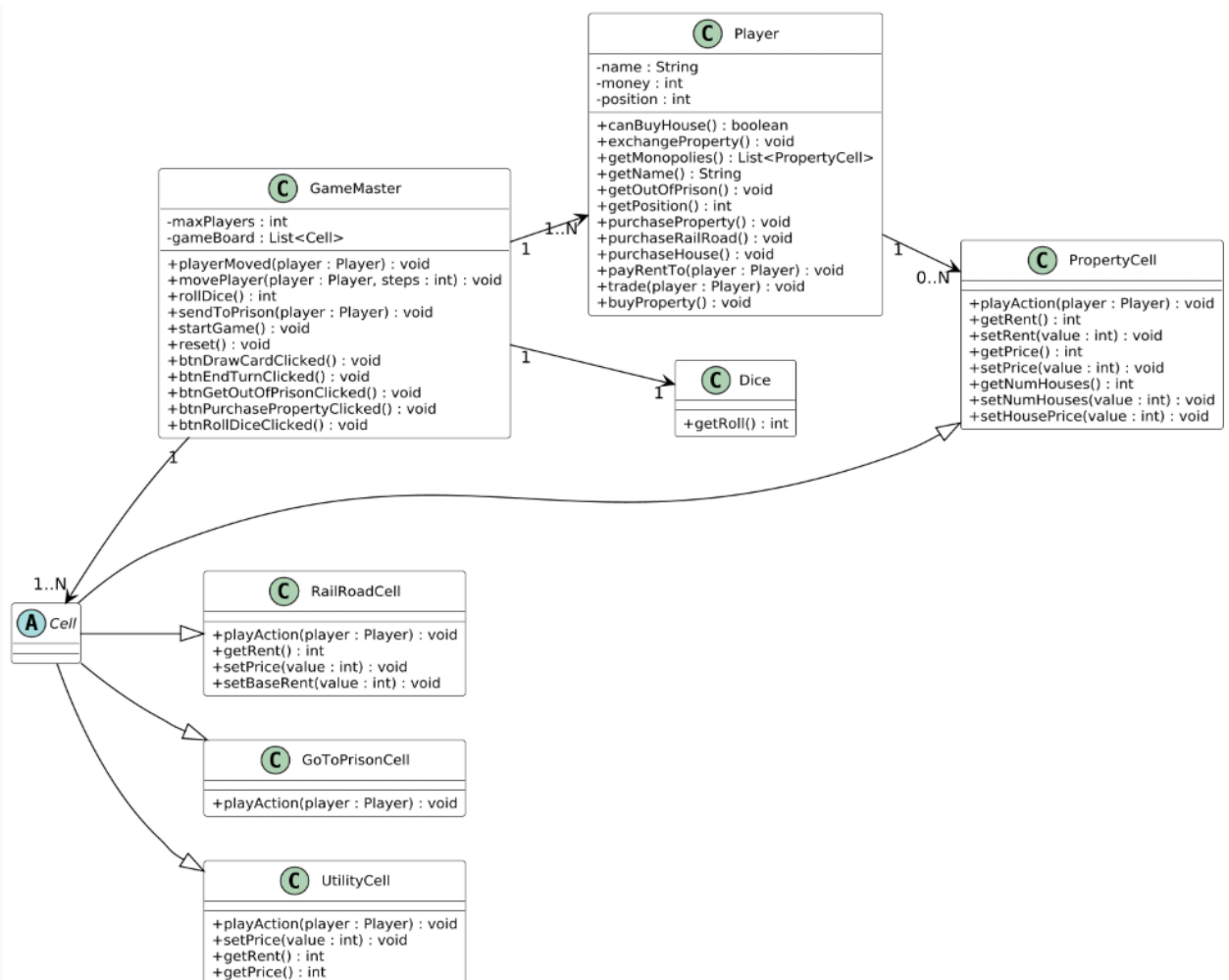


Рисунок 7 – Диаграмма классов

7 ДИАГРАММА СОСТОЯНИЙ

Диаграмма состояний (State Machine Diagram) – это UML диаграмма, которая описывает жизненный цикл объекта через последовательность состояний и переходов между ними. Каждое состояние представляет стабильную конфигурацию объекта в определенный момент времени, а переходы показывают, какие события или действия вызывают смену состояния. Диаграмма состояний помогает визуализировать, какие состояния может принимать объект, в каком порядке они меняются, и какие действия пользователя или системы вызывают эти переходы.

Нижеследующее описание приводится для диаграммы состояний статьи.

1. Начальное состояние

Система находится в состоянии инициализации.

На этом этапе приложение загружено, но игровой процесс ещё не начат. Пользователь не может выполнять игровые действия.

2. Состояние: Запрос количества игроков

Система отображает диалоговое окно «How many players?»

Пользователь должен ввести число участников и подтвердить ввод.

3. Состояние: Ввод имени игрока

После получения количества игроков система переходит в состояние последовательного ввода имён.

Отображается окно «Please input name for Player X», где X — номер текущего игрока.

Возможные переходы:

Пользователь вводит имя и подтверждает

– если это не последний игрок → повтор состояния для следующего игрока;

– если это последний игрок → переход к следующему состоянию.

4. Состояние: Переход к основному окну

После ввода всех имён система:

– создаёт игровые объекты;

– инициализирует поле;

– подготавливает интерфейс.

Затем происходит переход в главное игровое окно.

5. Состояние: Основное игровое окно

Система отображает:

– игровое поле;

– панели игроков;

– элементы управления ходом.

Пользователь может начинать игру.

Это состояние является основным рабочим и конечным состоянием приложения поскольку приложение завершило инициализацию и передало управление игровому процессу.

Описанная диаграмма приведена на рисунке 8.

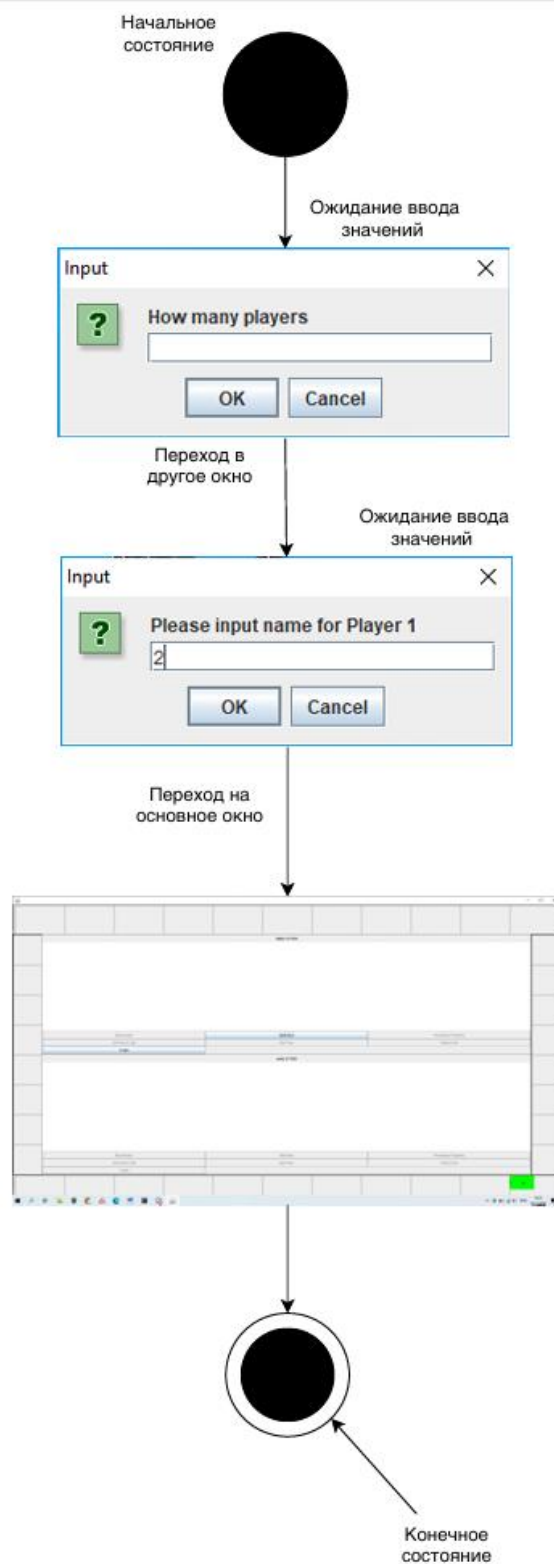


Рисунок 8 – Диаграмма состояний игры